

Orchestrator picker + interactive keys

ONE AGENT TO ROUTE THEM ALL

The installer starts at prompt 0 where you pick the AI agent orchestrator: Claude Code by default, or Cursor, Cline, Codex CLI, Aider, or a portable SYSTEM_PROMPT.md for anything else. It then writes the matching rules or hooks and, for non-Claude tools, copies the shipped skills. Prompt 7 completes install by asking for each provider's API key, printing where to get it, and writing a real 0600 .env file (falling back to a placeholder .env.example on skip).

What is new here: prompt 0 asks which orchestrator you use, then writes the right rules file (or hooks) and copies the bundled skills for non-Claude tools. Prompt 7 is new: for every provider you picked, it prints how to get an API key, reads the key with input hidden, and writes .env with 0600 permissions - or leaves a placeholder if you press ENTER to skip.

1. The orchestrator choice

c	claude-code (default)	writes ~/.claude/settings.json hooks + symlinks skills into ~/.claude/skills/
r	cursor	writes ~/.cursor/rules/sauron.mdc with alwaysApply:true; copies skills alongside
l	cline	writes ~/.clinerules (or ~/.clinerules); copies skills alongside
x	codex-cli	writes ~/.codex/instructions.md; copies skills into ~/.codex/sauron-skills/
a	aider	writes ~/.aider.sauron.md; add to .aider.conf.yml as read:
g	generic	writes ./SYSTEM_PROMPT.sauron.md you paste into any tool; copies skills alongside

2. The terminal session (Claude Code path)



interactive setup . one agent to route them all

0. Which agent orchestrator do you use?

- c) claude-code (default) full: hooks + Skill() invocations + delegate policy
 - r) cursor writes .cursor/rules/Framework.mdc (alwaysApply)
 - l) cline writes .clinerules with the delegate policy
 - x) codex-cli (OpenAI) writes ~/.codex/instructions.md
 - a) aider writes .aider.conf.yml + Framework.md conventions
 - g) generic / other writes SYSTEM_PROMPT.md you paste into any tool
- ✓ orchestrator: c

1. Where should it install?

- g) global → ~/.claude/settings.json (loads every session, every project)
- p) per-project → \$PWD/.claude/settings.json (only loads when Claude Code runs here)
- s) skip → print the settings JSON to stdout, do not write anything

2. Which skills should auto-load at session start?

These are invoked as the very first tool calls of every session.
Leave blank to skip a skill; type y to include it.

3. Which PAL models should be listed in the delegate-first policy?

Pick every model your PAL install actually has keys for.
Unselected models are dropped from the routing map so Claude never tries them.

Ready to write /tmp/sauron-pdf-v3/testhome/.claude/settings.json

Existing file will be backed up with .bak.<timestamp> suffix.

- ✓ wrote /tmp/sauron-pdf-v3/testhome/.claude/settings.json
- ! skipped skill symlinking; you must place skills under /tmp/sauron-pdf-v3/testhome/.claude/skills manually
- ! ~/.claude.json not found; make sure PAL is registered before starting Claude Code
- ✓ wrote env template: /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron.example

7. Enter API keys now to complete installation?

You can skip and edit .env.sauron.example later, or enter them now to have a real .env.sauron written with 0600 permissions.

Groq (report writing, validation)

How to get one:

1. Open <https://console.groq.com/keys>
2. Sign in with Google or GitHub
3. Click 'Create API Key', name it 'sauron'
4. Copy the key (starts with 'gsk_')

Free tier: gpt-oss-120b with ~500 rpm.

OpenRouter (nemotron / grok / or-free share one key; grok is your permissive high-context workhorse for security research)

How to get one:

1. Open <https://openrouter.ai/settings/keys>
2. Sign in with Google or GitHub

3. Click 'Create Key', name it 'sauron'
 4. Copy the key (starts with 'sk-or-v1-')
- Free-tier models (nemotron, grok-fast, meta-router) don't require credit.

Google Gemini (flash / pro share this key)

How to get one:

1. Open <https://aistudio.google.com/apikey>
2. Sign in with Google
3. Click 'Create API Key' in a new or existing Google Cloud project
4. Copy the key

Free tier: generous flash/pro quotas.

✓ wrote /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron (0600)

7b. Key entry summary

```
! Groq      placeholder (edit /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron to add it)
! OpenRouter placeholder (edit /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron to add it)
! Gemini     placeholder (edit /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron to add it)
```

Next steps

1. Register PAL as an MCP server in ~/.claude.json:

```
"mcpServers": { "pal": { "type": "stdio", "command": "/path/to/zen-mcp-server/.pal_venv/bin/python", "args": ["/path/to/zen-mcp-server/server.py"], "env": { ...keys... } } }
```
2. Source your API keys: `source /tmp/sauron-pdf-v3/testhome/.claude/.env.sauron` (after editing it)
3. Restart Claude Code.
4. On the next session start you should see the auto-invoked skills fire immediately.

✓ sauron setup complete.

3. What Claude sees at the next session start

The SessionStart hook fires *before* Claude responds. Its payload is injected into context.

SessionStart · additionalContext

MANDATORY FIRST ACTIONS: on session start and for EVERY model, BEFORE responding, immediately call the Skill tool for `caveman` (full mode), `pentesting-agent`, `validator` (preload). Full preference set: (1) delegate-first via PAL: groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. nemotron (nvidia, 1M ctx) for bulk reading of large files. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge. (2) KEEP IN CLAUDE: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, tool-sequence orchestration. (3) On every confirmed finding run the 4-step pipeline: validator skill, gap tests, adversarial challenge via mcp_pal_challenge (pro fallback groq), synthesize; delegate report writing to groq.

UserPromptSubmit · additionalContext (every message)

Reminder every message: delegate aggressively to PAL to conserve tokens (groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. nemotron (nvidia, 1M ctx) for bulk reading of large files. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge.). Keep in Claude ONLY: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, orchestration. FAILOVER: if any model refuses or errors, immediately re-route; a refusal is a routing problem, not a stop.

For non-Claude orchestrators: the same doctrine gets written into that tool's rules file (e.g. Cursor's .mdc, Cline's .clinerules). Since only Claude Code has a Skill() primitive, non-Claude installs also append a skills index to the rules file so the model knows what SKILL.md files it can read when a trigger phrase appears.

4. Files on disk after install

```
.claude/.env.sauron
.claude/.env.sauron.example
.claude/settings.json
```

.env.sauron.example (only the API keys your model picks need)

```
# sauron API keys - source this from your shell rc, or export before starting Claude Code.  
# Do NOT commit the real values.  
export GEMINI_API_KEY=your-gemini-key  
export OPENROUTER_API_KEY=your-openrouter-key  
export CUSTOM_API_URL=https://api.groq.com/openai/v1  
export CUSTOM_API_KEY=your-groq-key
```

Repo: github.com/crowx01/sauron · Sibling: [crowx01/let-there-be-light](https://github.com/crowx01/let-there-be-light) (dev-lifecycle)